

Lab0 实验报告

1. 认真阅读文档，学习 Git 的基本用法，并在报告中回答文档中的问题。

(1) 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

没有。（真的没有诶）

(2) 思考一下，Git 为什么要设计“暂存-提交”两个步骤？

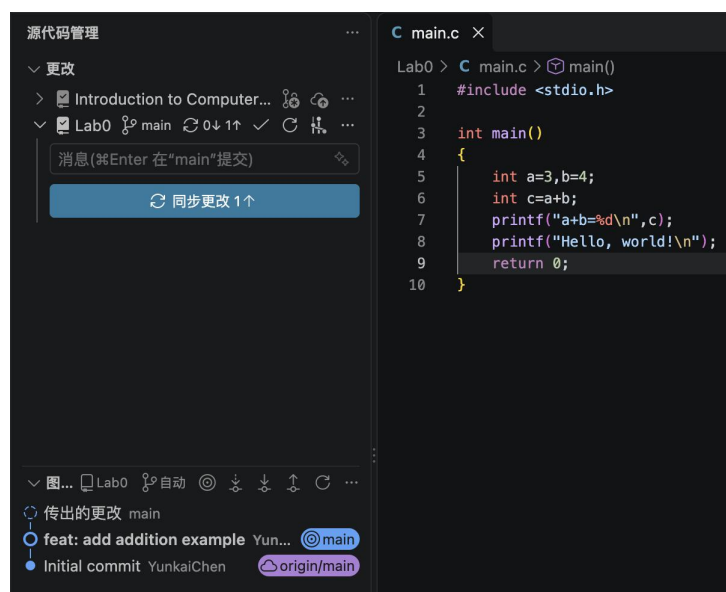
Git 设计“暂存-提交”两步，是为了让提交内容更可控。暂存区作为工作区和版本库之间的缓冲区，允许我们挑选部分改动提交、分多次有逻辑地提交，同时也能临时搁置没改完的部分。如果没有暂存区，每次提交只能把当前所有改动直接提交，提交历史会变得混乱、难以维护。

(3) git branch 和 git branch-a 的区别是什么？查阅资料并回答。

git branch 只显示本地分支；git branch-a 会同时显示本地分支和远程分支，远程分支以 remotes/origin/xxx 的形式列出。前者用于查看本地有哪些分支，后者还能看到远程仓库的分支情况，方便决定是否要 checkout 或拉取某个远程分支。

2. 使用我们的模板仓库建立个人仓库，完成 main.c 文件中的 TODO 部分并进行一次 commit。

在 main.c 中添加了一个简单的加法运算并输出。



3. 在下面的三个网页中任选其二进行阅读，简要概括其内容，并谈谈你对“为什么要学习 Git”这个问题的理解。

第一篇文章介绍了 Commit message 的规范。每次提交都要写说明，正文要写详细动机，页脚用于标记不兼容变动或关闭 issue。规范化的好处是：方便浏览历史、过滤查找、自动生成，文章还介绍了 Commitizen 等配套工具。

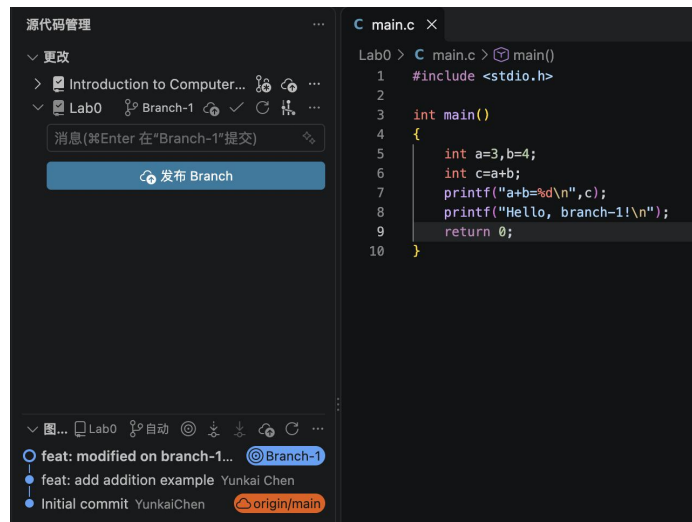
第二篇文章介绍了 Gitflow 分支管理策略。核心是几条分支各司其职：master 对应线上稳定版本、develop 是开发主干、feature 开发新功能、release 用于测试、hotfix 紧急修线上 bug。核心原则是“从哪里来，回到哪里去”，保证不同分支互不干扰，避免多人协作的冲突和版本混乱。

从这两篇文章可以看出，Git 不只是存代码的工具，而是一整套协作和版本管理的方法论。规范的 commit message 让提交历史清晰可读，别人（以及未来的自己）能快速看懂

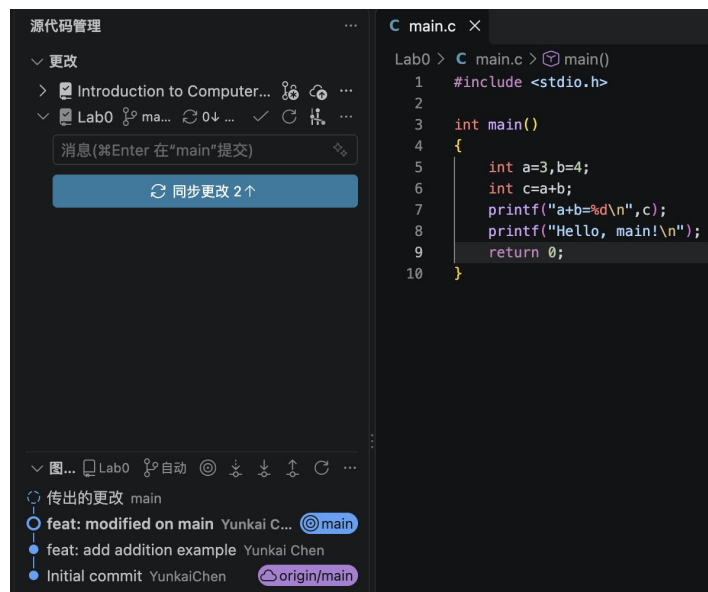
每次改动；Gitflow 这样的分支策略则让多人协作有条不紊，避免代码冲突和版本混乱。在真实的软件开发和团队协作中，这些规范决定了项目能不能高效、可靠地推进。学习 Git，本质上是学习如何与他人协作、如何管理复杂的代码演进。

4. 学习 Git 分支管理，新建 feature 分支，在该分支以及 main 分支上对 main.c 分别进行一次修改与提交。随后将 feature 分支 merge 到 main 分支（即切换回 main 分支执行 git merge feature），并处理发生的合并冲突。

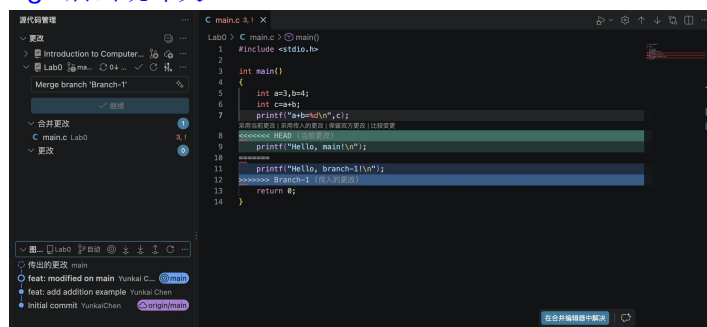
新建分支 branch-1，将其中的“Hello, world!” 输出语句修改为“Hello, branch-1!”



返回 main，将相同的语句修改为“Hello, main!”



执行 git merge 后出现冲突:



选择“采用当前更改”（即使用 **main** 中的更改），完成合并：

